

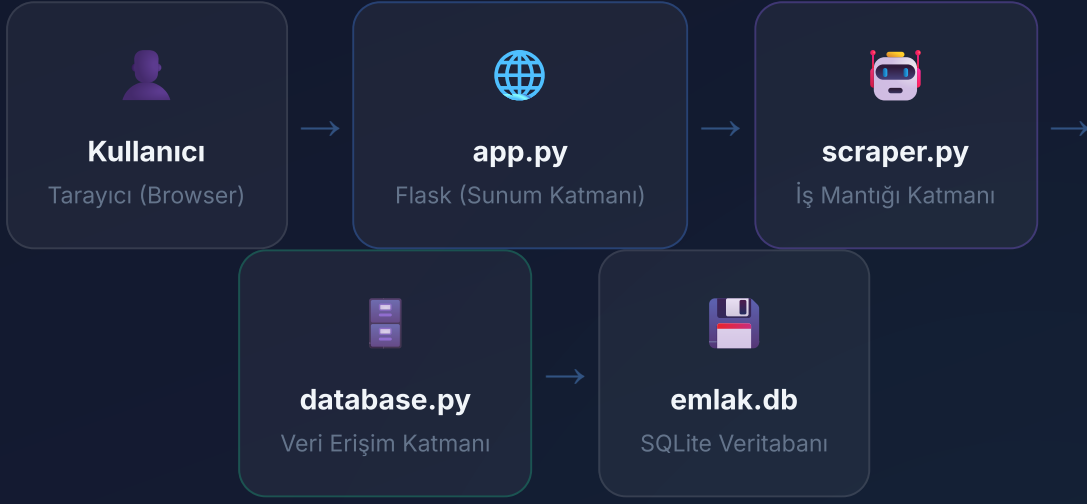


Emlak Avcısı Pro

Teknik Mimari & Kod Analizi Sunumu

Bitirme Projesi — Teknik Detaylar — 2026

3 Katmanlı Mimari Yapı



Sunum Katmanı (app.py)

Flask web framework üzerine inşa edilmiştir. HTTP GET/POST isteklerini yönetir, Jinja2 şablon motoruyla HTML render eder. AJAX ile bot durumunu asenkron olarak frontend'e bildirir.

İş Mantığı Katmanı (scraper.py)

SeleniumBase (Undetected ChromeDriver) ile web sayfalarını otonom olarak gezer. BeautifulSoup ile HTML ayrıştırma yapar. Veriyi yapılandırıp veritabanına yazar.

Veri Erişim Katmanı (database.py)

SQLite veritabanı şemasını tanımlar ve otomatik oluşturur. 30 sütunluk ilişkisel tablo yapısı. ilan_no TEXT UNIQUE ile duplikasyon kontrolü.

database.py — Veritabanı Modülü

```
import sqlite3, os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

def init_db():
    conn = sqlite3.connect(os.path.join(BASE_DIR, "emlak.db"))
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS ilanlar (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            ilan_no TEXT UNIQUE,      -- Duplikasyon engeli
            baslik TEXT, fiyat TEXT, link TEXT,
            resim_yolu TEXT, aciklama TEXT, lokasyon TEXT,
            ilan_tarihi TEXT, emlak_tipi TEXT,
            m2_brut TEXT, m2_net TEXT, oda_sayisi TEXT,
            bina_yasi TEXT, bulundu_kat TEXT,
            ...                        -- Toplam 30 sütun
            tarih DATETIME DEFAULT CURRENT_TIMESTAMP
        )
    """)
    conn.commit()
    conn.close()
```

🔑 Tasarım Kararları

- **UNIQUE Kısıtlama:** `ilan_no TEXT UNIQUE` ile aynı ilanın tekrar eklenmesi SQL seviyesinde engellenir.
- **AUTOINCREMENT:** Her kayıt benzersiz artan ID alır, sıralama ve referans kolaylığı sağlar.
- **CURRENT_TIMESTAMP:** Kaydın sisteme eklenme zamanı otomatik yazılır.
- **CREATE IF NOT EXISTS:** Uygulama her başlatıldığında çağrılır, tablo varsa atlar — idempotent yapı.

Neden SQLite? Harici sunucu gerektirmez, tek dosya (emlak.db), Python'da built-in destek, kurulum sıfır.

scraper.py — Cloudflare Bypass Mekanizması

```
scraper.py - Bot Başlatma
# Undetected ChromeDriver ile Cloudflare geçişi
with SB(uc=True, test=True, ad_block_on=True,
        binary_location=BRAVE_PATH) as sb:

    sb.maximize_window()

    # Sayfa açılışı + rastgele bekleme
    sb.open(guncel_link)
    random_sleep(5, 7) # Bot tespitini önler

    # Her 5 ilandan sonra çerezleri temizle
    if index > 0 and index % 5 == 0:
        sb.delete_all_cookies()
        random_sleep(1, 2)
```

🛡️ Anti-Bot Stratejileri

- 1. uc=True:** Undetected ChromeDriver modu aktif — tarayıcı parmak izi maskelenir.
- 2. random_sleep():** Her istekte 2-7 sn arası rastgele gecikme — insan davranışı simülasyonu.
- 3. Cookie Temizliği:** Her 5 ilandan sonra çerez silinerek oturum takibi kırılır.
- 4. ad_block_on:** Reklam / izleme scriptlerini engeller — hız + gizlilik.

```
scraper.py - Rastgele Gecikme Fonksiyonu
def random_sleep(min_s, max_s):
    """İnsan benzeri bekleme - bot tespitini zorlaştırır"""
    time.sleep(random.uniform(min_s, max_s)) # Her seferinde farklı süre
```

Veri Çıkarma & Özellik Haritası

```
scraper.py – Özellik Haritası (Dictionary Mapping)
# Sitedeki HTML etiketlerini DB sütunlarıyla eşleştiren sözlük

özellik_haritası = {
    "İlan No": "ilan_no",
    "İlan Tarihi": "ilan_tarihi",
    "Emlak Tipi": "emlak_tipi",
    "m² (Brüt)": "m2_brut",
    "Oda Sayısı": "oda_sayisi",
    "Bina Yaşı": "bina_yasi",
    ... # 24 alan daha
}

# Varsayılan değerlerle veri paketini hazırla
veri_paketi = {col: "-" for col in özellik_haritası.values()}

# HTML'den li etiketlerini tara, haritadaki anahtarı bul
for li in info_ul.find_all("li"):
    etiket = li.find("strong").text.strip()
    deger = li.find("span").text.strip()
    if etiket in özellik_haritası:
        veri_paketi[özellik_haritası[etiket]] = deger
```

Tasarım

Deseni:

Dictionary Mapping

Website'deki Türkçe etiket adları ("Oda Sayısı") ile veritabanı sütun adlarını ("oda_sayisi") eşleştiren bir **sözlük haritası** kullanılır.

Avantajları:

- Yeni alan eklemek 1 satır — ölçeklenebilir
- if-elif zinciri yerine O(1) sözlük araması
- Varsayılan değer ("-") ile eksik veri koruması



BeautifulSoup Parsing

find() ve find_all() metodları ile DOM ağacı traversal. CSS sınıf adları ve

In-Memory Set & Sayfalama Algoritması

```
scraper.py - In-Memory Set (RAM Cache)
# Tüm mevcut ilan ID'lerini RAM'e yükle

mevcut_ilanlar = set()
cursor.execute("SELECT ilan_no FROM ilanlar")
for sat in cursor.fetchall():
    mevcut_ilanlar.add(sat[0])

# Her ilan için DB'ye sormak yerine Set'te kontrol
if ilan_id in mevcut_ilanlar: # O(1) zaman
    continue # Zaten var, atla
```

⚡ Neden Set Kullanıldı?

Problem: 500 ilan tararken her biri için `SELECT COUNT(*) WHERE ilan_no=?` sorgusu = 500 DB bağlantısı.

Çözüm: Başta tüm ID'leri bir Set'e alıp in operatörü ile $O(1)$ sabitinde kontrol. ~100x hız kazanımı.

```
scraper.py - URL Tabanlı Sayfalama
su_anki_sayfa = 1
MAX_SAYFA = (limit // 20) + 1

while su_anki_sayfa <= MAX_SAYFA:
    offset = (su_anki_sayfa - 1) * 20

    # URL'ye otomatik pagination parametre
    if "?" in hedef_link:
        guncel_link = f"{hedef_link}&page={su_anki_sayfa}"
    else:
        guncel_link = f"{hedef_link}?page={su_anki_sayfa}"

    sb.open(guncel_link)
    su_anki_sayfa += 1
```

📄 Sayfalama Mantiğı

Sahibinden sayfada 20 ilan gösterir. `pagingOffset` parametresiyle sonraki sayfaya geçilir. URL'de ? var mı kontrol edilip uygun ayırıcı (& veya ?) kullanılır.

app.py — Flask Backend Mimarisi

```

app.py — Dinamik Filtreleme & Sıralama
# Dinamik SQL sorgusu oluşturma (Query Builder)

query_conditions = "1=1"      # Her zaman doğru — base koşul
params = []

if lokasyon_filtre:
    query_conditions += " AND lokasyon = ?"
    params.append(lokasyon_filtre) # Parameterized!

if fiyat_araligi == "2M-5M":
    query_conditions += """ AND CAST(
        REPLACE(REPLACE(fiyat, '.', ''), ' TL', '')
        AS INTEGER) BETWEEN 2000000 AND 5000000 """

# Sıralama seçeneği
order_clause = "tarih DESC" # Varsayılan
if siralama == "fiyat_artan":
    order_clause = "CAST(REPLACE( ... ) AS INTEGER) ASC"

# Sayfalama (Pagination) — LIMIT + OFFSET
query = f"SELECT * FROM ilanlar WHERE {query_conditions}
        ORDER BY {order_clause} LIMIT ? OFFSET ?"
params.extend([per_page, (page-1) * per_page])

```

🔧 Query

Builder Deseni

"1=1" Tekniği:

Koşul zincirinin başına yerleştirilir, böylece her yeni filtre AND ile eklenebilir — ilk koşul sorunu ortadan kalkar.

Parameterized

Query (?):

Kullanıcı girdileri asla SQL dizisine gömülmez — SQL Injection koruması.

CAST +

REPLACE:

Fiyatlar "2.500.000 TL" formatında saklanır. Sıralama için nokta ve "TL" kaldırılıp INTEGER'a çevrilir.

Thread & AJAX Mimarisi

```

app.py - Arka Plan Thread
# Bot durumu - paylaşımlı sözlük
bot_durumu = {"mesaj": "", "calisiyor": False}

def botu_arka_planda_calistir(link, limit):
    bot_durumu["calisiyor"] = True
    try:
        sonuc = botu_calistir(link, limit)
        bot_durumu["mesaj"] = sonuc
    finally:
        bot_durumu["calisiyor"] = False

# Ana thread'i bloklamadan yeni Thread başlat
t = threading.Thread(target=botu_arka_plan
                      args=(link, limit))

t.daemon = True
t.start()

```

```

index.html - AJAX Polling (Frontend)
// Her 2.5 saniyede Flask'tan durum sorgula

function checkBotStatus() {
    fetch('/status')
        .then(r => r.json())
        .then(data => {
            if(data.calisiyor) {
                botStatusWin.style.display
                botMsg.textContent = data.
            } else {
                botStatusWin.style.display
                if(wasRunning) window.loca
            }
        });
}

setInterval(checkBotStatus, 2500);

```

Thread Tasarımı

daemon=True: Ana uygulama kapanırsa thread de sonlanır — zombie process önlemi.

Neden Thread? Bot 5-30 dakika çalışabilir. Thread kullanılmazsa web sunucu o süre boyunca kitlenir.

📡 AJAX Polling Stratejisi

/status endpoint: jsonify(bot_durumu) ile JSON yanıt döner.

Auto-reload: Bot durduğunda wasRunning flag'i ile otomatik sayfa yenileme — yeni ilanlar anında görüntülenir.

Güvenlik Önlemleri & UI Teknikleri



Güvenlik Katmanları

TEHDİT	YÖNTEM	KOD
SQL Injection	Parameterized ?	"WHERE x = ?", (val,)
Path Traversal	secure_filename	secure_filename(id)
POST Tekrarı	PRG Pattern	return redirect(url_for(.primary, --glass-bg) ile tema yönetimi.
XSS	Jinja2 Auto-Escape	{{ ilan.baslik }}
Dosya Silme	Mutlak Yol	os.path.join(BASE, id)



Frontend Teknikleri

Glassmorphism CSS

backdrop-filter: blur(16px), rgba()
(val,)
arka planlar, border: 1px solid
rgba(255,255,255,0.1)

CSS Custom Properties

:root üzerinde tanımlanan değişkenler (-
primary, --glass-bg) ile tema yönetimi.
prefers-color-scheme: dark ile
otomatik karanlık mod.

Lightbox & Lazy Loading

Resimlere tıklanınca position:fixed
overlay açılır. loading="lazy" ile
görseller sadece görünür alanda yüklenir.

Olası Sorular & Cevaplar

S: Neden SQLite tercih ettiniz, MySQL veya PostgreSQL kullanmadınız?

C: SQLite kurulum gerektirmez (zero-config), tek bir dosyada (emlak.db) çalışır ve Python'da built-in desteklidir. Proje yerel bir masaüstü uygulaması olduğundan yüksek eşzamanlı bağlantı ihtiyacı yoktur. Üretime taşınması gerekirse SQLAlchemy ORM ile veritabanı motoru kolayca değiştirilebilir.

S: Cloudflare engelini nasıl aşıyorsunuz, bu yasal mı?

C: SeleniumBase kütüphanesinin "Undetected ChromeDriver" (uc=True) modu, otomatik tarayıcı parmak izini maskeleyerek bot algılama sistemlerini aşar. Ayrıca random_sleep() ile insan benzeri bekleme süreleri ve çerez temizleme stratejileri uygulanır. Proje eğitim ve araştırma amaçlıdır; ticari kullanım hedeflenmemektedir.

S: Thread kullanımında race condition riski yok mu?

C: bot_durumu sözlüğü sadece tek bir thread (arka plan bot thread'i) tarafından yazılır, ana thread sadece okur. Python'da GIL (Global Interpreter Lock) sayesinde basit sözlük okuma/yazma işlemleri atomiktir. Ayrıca aynı anda birden fazla bot başlatılmasını kontrol eden if not bot_durumu["calisiyor"] kilidi mevcuttur.

S: SQL Injection'a karşı nasıl korunuyorsunuz?

C: Tüm kullanıcı girdileri (lokasyon, arama terimi, ilan numarası) doğrudan SQL stringine eklenmez; bunun yerine parameterized query (?-placeholder) kullanılır. Örneğin: cursor.execute("WHERE lokasyon = ?", (user_input,)). Bu yöntem, sqlite3 modülünün kendi escape mekanizmasını devreye sokar.

S: "1=1" koşulu ne işe yarar, güvenlik sorunu oluşturmaz mı?

C: "1=1" bir Query Builder desenidir. Filtrelerin dinamik olarak AND ile eklenmesini kolaylaştırır (ilk koşulda AND yazma sorununu ortadan kaldırır). Bu koşul her zaman doğru olduğundan sonucu değiştirmez. Güvenlik riski oluşturmaz çünkü sabit bir string olup kullanıcı girdisi içermez.

S: Fiyat sıralamasında CAST+REPLACE neden gerekiyor?

C: Fiyatlar veritabanında "2.500.000 TL" gibi TEXT formatında saklanır. Sayısal sıralama yapabilmek için önce nokta ve "TL" ifadesi REPLACE ile kaldırılır, ardından CAST ile INTEGER'a çevrilir. Bu yaklaşım veri bütünlüğünü korurken sıralama esnekliği sağlar.